

מעבדה 12: בעיות סנכרון קלאסיות – בעיית הקוראים והכותבים בעדיפות כותב (Readers – Writers)

הקדמה ומטרות

במעבדות הקודמות חקרנו בעיות סנכרון שבהן ישויות הקצה ביצעו פעולות סימטריות או פעולות קבועות של העברת נתונים (כמו יצרן וצרכן). עם זאת, במערכות הפעלה רבות, הגישה למשאב משותף (כגון בסיס נתונים בזיכרון, טבלת ניתוב של הליבה או מערכת קבצים) מתאפיינת בשני סוגי ישויות בעלות דפוסי התנהגות שונים לחלוטין:

1. **הקוראים Readers** – ישויות המבקשות לקרוא נתונים בלבד מתוך המשאב. קוראים אינם משנים את המידע, ולכן אין מניעה שמספר קוראים ייגשו למשאב בו-זמנית (קריאה מקבילית בטוחה).
2. **הכותבים Writers** – ישויות המבקשות לעדכן, למחוק או לכתוב מידע חדש למשאב. פעולת הכתיבה משנה את מצב המערכת, ולכן בזמן שכותב פועל, **חל איסור על שום ישות אחרת (קורא או כותב נוסף) לשהות במשאב** (מניעה הדדית מוחלטת).

בעיה קלאסית זו הוגדרה בהרצאה האחרונה. פתרון נאיבי הנותן קדימות לקוראים עלול להוביל לבעיה מערכתית קשה המכונה "הרעבה" של כותבים: אם זרם הקוראים למערכת הוא בלתי פוסק, המשאב לעולם לא יתפנה, וחוטי הכותבים ימתינו בשינה לנצח.

במעבדה זו נממש את **הווריאציה השנייה של הבעיה (Writers-Priority Readers-Writers)**. במודל זה, ברגע שחוט כותב מודיע שהוא מעוניין לגשת למשאב, קוראים חדשים המגיעים למערכת נחסמים וממתינים בחוץ, כדי לאפשר לכותב לקבל את המשאב במהירות האפשרית ברגע שהקוראים הנוכחיים מסיימים.

מטרות המעבדה

- הבנה עמוקה של דפוסי סנכרון א-סימטריים מורכבים במערכות הפעלה.
- שליטה בניהול מערך סמפורים של POSIX לפתרון תופעת ההרעבה וקביעת עדיפויות בתוכנה.
- יישום מנגנוני מעקב ומונים מוגנים (Read Count ו-Write Count) לניהול מצב המשאב הגלובלי.
- הבנה מעשית של תופעות Deadlock הנגרמות מהצלבת חסימות בין קוראים לכותבים.

מבנה הנתונים והקבצים

המעבדה מנוהלת כסימולציית עומס רב-חוטית מלאה. התוכנית תקבל שני ארגומנטים בשורת הפקודה: כמות חוטי הקוראים שיש ליצור, וכמות חוטי הכותבים שיש ליצור.

פונקציות הליבה של סמפור ב Linux

- יוקצו דינמית חוטים עבור הקוראים וחוטים עבור הכותבים לפי דרישת המשתמש.
- המשאב המשותף המוגן יהיה משתנה שלם גלובלי פשוט המייצג בסיס נתונים (`shared_database`).
- המערכת תנהל מונים גלובליים למעקב אחר כמות הקוראים השוהים כעת בפועל בתוך בסיס הנתונים, וכמות הכותבים הממתינים או פועלים במערכת. עדכון מונים אלו מחייב הגנה קפדנית באמצעות סמפורים בינאריים ייעודיים.

דגשים והסברים

כדי לאפשר לכם להשלים את המשימה בהצלחה, להלן פירוט של הכלים האטומיים, והלוגיקה המערכתית הנדרשת למימוש הבעיה בסביבת Linux באמצעות הספרייה `< semaphore.h >`

לוגיקת המניעה הא-סימטרית בעדיפות כותב

כדי לממש עדיפות לכותבים ולמנוע מהם הרעבה, לא ניתן להסתפק במבנה הפשוט שהוצג בהרצאה. יש צורך בניהול מתוחכם יותר המבוסס על מערך של מספר סמפורים בינאריים ומונים:

1. מוני המצב:

a. `read_count`: מונה שלם העוקב אחר מספק חוטי הקוראים שנמצאים כעת בתוך האזור הקריטי של קריאת המצב.

b. `write_count`: מונה שלם העוקב אחר מספר חוטי הכותבים שנמצאים כעת במערכת (בין אם הם ממתינים בתור לכתוב ובין אם אחד מהם כותב בפועל).

2. הסמפורים הבינאריים (Mutex):

a. `mutex_r`: סמפור בינארי (ערך התחלתי 1) המגן בלעדית על האזור הקריטי של עדכון המשתנה `read_count`.

b. `mutex_w`: סמפור בינארי (ערך התחלתי 1) המגן בלעדית על האזור הקריטי של עדכון המשתנה `write_count`.

c. `queue_block`: סמפור בינארי (ערך התחלתי 1) המשמש כמחסום שער גלובלי. זהו לב המנגנון לעדיפות כותבים: קוראים מחויבים לעבור דרכו לפני שהם ניגשים למשאב. ברגע שכותב מגיע, הוא תופס/חוסם את הסמפור הזה, ובכך מונע מקוראים חדשים להצטרף לתור ומאלץ אותם לישון בחוץ.

d. `resource_access`: סמפור בינארי (ערך התחלתי 1) המגן פיזית על הגישה הישירה למשאב המשותף (`shared_database`).

פסאודו-קוד מילולי של פרוטוקול הסנכרון

עליכם לתרגם את העקרונות המילוליים הבאים לקוד C מדויק בתוך פונקציות הגישה:

פרוטוקול חוט קורא:

1. **שלב המחסום:** הקורא מנסה לעבור דרך סמפור השער `queue_block` (נחסם אם יש כותב שמחזיק בשער).
2. **שלב הרישום:** הקורא תופס את `mutex_r`, מעלה את המונה `read_count` ב-1. אם הוא הקורא הראשון שמגיע, הוא מחויב לנסות לנעול את סמפור המשאב `resource_access` כדי לוודא שאין כותב בפנים ולמנוע מכותבים להיכנס. לאחר מכן הוא משחרר את `mutex_r`.
3. **שלב הפינוי:** הקורא משחרר מיד את סמפור השער כדי לאפשר לקוראים נוספים בתור לעבור.
4. **[אזור קריטי: ביצוע קריאה מהמשתנה המשותף והדפסת הערך].**
5. **שלב העזיבה:** הקורא תופס של `mutex_r`, ומוריד את המונה `read_count` ב-1. אם הוא הקורא האחרון שיוצא מהמשאב, הוא מחויב לשחרר את סמפור המשאב `resource_access` כדי לאפשר לכותבים הממתינים בחוץ להיכנס. לבסוף הוא משחרר את הסמפור שברשותו.

פרוטוקול חוט כותב:

1. **שלב הרישום:** הכותב תופס את `mutex_w`, ומעלה את המונה `write_count` ב-1. אם הוא הכותב הראשון שמגיע ומכריז על כוונה לכתוב, הוא מחויב לנעול/לתפוס את סמפור השער, ובכך הוא סוגר את השער ומונע מקוראים חדשים להיכנס למערכת. לא מכן הוא משחרר את הסמפור שברשותו.
2. **שלב הנעילה המוחלטת:** הכותב מנסה לנעול את סמפור המשאב `resource_access`. הוא ייחסם כאן אם ישנם קוראים קודמים שעדיין שוהים בפנים, וימתין עד שהקורא האחרון שבהם יצא וישחרר את המשאב.
3. **[אזור קריטי: ביצוע כתיבה/עדכון של המשתנה המשותף].**
4. **שלב שחרור המשאב:** הכותב משחרר את סמפור המשאב `resource_access`.
5. **שלב העזיבה והפתיחה:** הכותב תופס את `mutex_w`, ומוריד את המונה `write_count` ב-1. אם הוא הכותב האחרון שעוזב את המערכת ואין יותר כותב שממתין בתור, הוא מחייב לשחרר את סמפור השער `queue_block`, ובכך הוא פותח מחדש את השער ומאפשר לכל חוט קורא שהמתין בחוץ להתעורר ולהיכנס. לבסוף הוא משחרר את הסמפור שברשותו.

משימות לביצוע

משימה 1: הגדרת קובץ הכותרת *pipeline.h*

עליכם להגדיר בעצמכם את מבני הנתונים ואת חתימות הפונקציות הנדרשות. עליכם ליצור מבנה מרכזי יחיד:

1. *SharedResource*: מבנה נתונים המאגד בתוכו את המשאב המשותף, את המונים השלמים של מצב המערכ, ואת ארבעת הסמפורים הנדרשים לניהול המחסום והמניעה ההדדית.

וכן את חתימות הפונקציות למימוש:

```
void init_resource(SharedResource *sr);
void* reader_worker(void *arg);
void* writer_worker(void *arg);
```

משימה 2: מימוש לוגיקת הסמפורים והבאפר המעגלי *pipeline.c*

עליכם לממש את פונקציות הניהול בקובץ זה:

1. *init_resource*: תאתחל את המשתנה הגלובלי ל-0, את המונים ל-0 ותאתחל את ארבעת הסמפורים.
2. *reader_worker*: פונקציית העבודה של הקוראים. כל חוט ירוץ בלולאה קצרה (למשל 3 איטרציות). בכל איטרציה עליו לבצע את פרוטוקול הכניסה של הקורא, לקרוא ולהדפיס את הערך של המשאב המשותף, לבצע את פרוטוקול העזיבה, ולהשהות את הריצה באמצעות *usleep*.
3. *writer_worker*: פונקציית העבודה של הכותבים. כל חוט ירוץ בלולאה קצרה. בכל איטרציה עליו לבצע את פרוטוקול הכניסה והחסימה של הכותב, לעדכן את המשתנה הגלובלי (להוסיף לו 1), לבצע את פרוטוקול העזיבה והשחרור, ולהשהות את הריצה באמצעות *usleep*.

משימה 3: שימוש בקובץ הדרייבר הראשי *main.c*

עליכם להשתמש בקובץ ה- *main.c* שסופק לכם מראש. קובץ זה קורא את הארגומנטים, מקצה דינמית את מערכי החוסים, מפעיל את כולם במקביל, וממתין לסיומם בצורה מסודרת. **אין לבצע שינויים בקובץ זה.**

מה להגיש ובאיזה פורמט

יש להגיש קובץ ZIP בשם *Lab12_ID1_ID2.zip* (החליפו את ID בתעודת הזהות שלכם) על הקובץ להכיל את הקבצים:

1. *pipeline.c*

2. *pipeline.h*

3. *main.c*

4. *Makefile* – קובץ בנייה המקמפל עם הדגלים *pthread* – *Wextra* – *Wall*

אי הקפדה על הנחיות, כולל פורמט הגשה הדיגיטלי, תגרור הורדה בציון התרגיל.